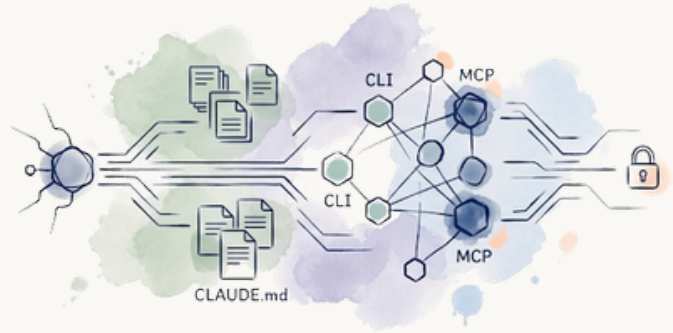


Claude *Token* Savings Guide

Context Hygiene · Principle-Based · TechLetter.co



Tokens are Claude's working memory. Inputs, files, tools, and chat history all pile into the same space. When that space stays clean, Claude performs better and costs less. This guide offers principles, not prices. Applicable today and tomorrow.

Core principle: When context fills up, Claude "forgets the middle." It's not the size of the limit, it's what goes into it.

1. Compress output

By default, Claude writes long, polite responses. Changing this costs nothing.

Caveman rules: Adding "short answers, no filler, tools before explaining, max 3 sentences" to CLAUDE.md can cut output by more than half.

Token-efficient CLAUDE.md: Drop the file from drona23/claude-token-efficient into the project root. Disables verbosity in one step.

Sweet spot: A CLAUDE.md of around 500-800 tokens, composed of clear short rules, tends to be the most effective.

2. Clean input

CLAUDE.md is re-read with every message. If it's 1,000 lines, even "hello" loads all 1,000 lines.

CLAUDE.md: keep it short

Keep it under 200 lines. Only rules, tech stack, key file refs. Details in separate files; Claude reads on demand.

.claudeignore: drop the noise

Exclude node_modules, dist, build, .git, logs, lock files, minified files, assets. Tools like claude_ignore_optimizer automate this.

4 files, minimal setup: CLAUDE.md + .claudeignore + a few small info files. Call topic-specific files on demand instead of loading upfront.

3. Tiered context and project structure

Rather than loading everything at once, layering information and opening only what's needed is more effective.

Tier 1 · Always

Project purpose, CLAUDE.md rules, tech stack. Loaded every session.

Tier 2 · On demand

Code files, docs, API specs. Called with @ for targeted access.

Tier 3 · Link only

Large references, wikis, long PDFs. Link or summarize instead.

4. Model strategy and thinking budget

Using the most powerful model for every task is like taking a taxi everywhere.

Haiku

- File reading, lookups
- Subagent tasks
- Running tests
- Lowest cost

Sonnet

- General coding, debug
- Handles ~80% of work
- Near-Opus quality
- Much more affordable

Opus

- Complex architecture
- Multi-file refactoring
- Critical code review
- Highest cost

Thinking budget Keeping it low makes sense. Use /effort to reduce for simple tasks. Extended thinking can consume tens of thousands of output tokens. Reserve full budget for genuinely hard problems.

5. MCP or CLI? + Daily practices

Critical Every connected MCP server reloads tool definitions with every message. This is ongoing, not one-time. When a CLI exists, it's more efficient.

Tool	MCP server	CLI alternative
Figma	Dumps entire file tree	figma-cli: fetches only queried element
GitHub	Bulk-loads issues, PRs	gh CLI: targeted, much lighter
Search	Multi-tool MCP overhead	rg/grep + bash: zero overhead

Principle: When a CLI exists, prefer it. When MCP is necessary, keep at most 1-2 servers active.

Governance note: Every MCP/CLI can expose company data to third parties. Know what's connected, keep permissions minimal.

Daily practices

1 /clear after each task

Fresh start prevents context bleed.

→ Clean slate, clear thinking

2 /compact at 60-70%

Manual compact at logical breakpoints.

→ /compact "preserve code"

3 Targeted @ reads

@src/api/auth.js, not 'find my auth.'

→ @file > glob scanning

4 Model switching

/model for task-based switching.

→ /model sonnet

5 Prompt caching

Cache recurring instructions.

→ cache_control

6 Plan first, code later

Validate before writing.

→ 95% confidence first

Token diet in 30 minutes

P.S. Skip /clear when your next task is small and related, the context is already loaded, reusing it costs nothing. Clear for topic switches, not every task boundary.

Can be applied today, results visible by tomorrow.

- 1** Trim CLAUDE.md to under 200 lines, move details to separate files.

2 Create a .claudeignore, disable unused MCP servers.

3 Set the default model to Sonnet, keep the thinking budget low.

4 Add caveman rules or token-efficient CLAUDE.md to the project.

5 Build /clear and /compact habits; check for CLI alternatives.

6 Measure with /cost after one week, see the difference.

Github-Toolbox

Output compression

JuliusBrussee/caveman
drona23/claude-token-efficient

Context management

affaan-m/everything-claude-code
danielrosehill/Claude-Code-Context-Toolkit
CodeRod-Solutions/claude_ignore_optimizer
nadimtuhin/claude-token-optimizer

MCP alternatives

silships/figma-cli
mksglu/context-mode
eusebioresende/pitlane-mcp
henkisdabro/MCP-Server-Selector

Minimal output mode skill.
Drop-in CLAUDE.md, cuts verbose output.

Token optimization + skills reference.
CLAUDE.md + CONTEXT.md separation toolkit.
Automatic .claudeignore generator.
4-file minimal setup.

Lightweight Figma MCP alternative.
Sandboxes MCP output, sends only summaries.
Local, lightweight-schema token-efficient MCP.
Toggle servers on/off by task.

Measurement

jerhadt/token-counter
Local token counter CLI.
alexgreensh/token-optimizer
Ghost token audit and quality scoring.

P.S. All tools in the toolkit are GitHub repos — most can be installed directly from within Claude Code via the plugin marketplace (no manual cloning needed). token-optimizer and context-mode are already listed there. For the rest, a quick git clone and you're set.